Homework Handout — The BrewHub Case Study

BrewHub is a fictional multi-hub coffee-roasting and ordering platform — think of it as a bigger, distributed cousin of Starbuzz Coffee (Chapter 4) and the ChocolateBoiler factory floor (Chapter 5). Across five milestones, students design one growing codebase, applying a different pattern from Chapters 1 through 5 at each stage.

Suggested submission format: one Java package per milestone, plus a short README per milestone explaining the design choice — this is where students demonstrate they understand *why* the pattern was needed, not just that the code compiles.

Milestone 1 — Strategy: Pricing & Loyalty Strategies

Case Study

BrewHub wants to support several discount schemes at checkout — a student discount, happy-hour pricing, loyalty-tier pricing, and "no discount" — and new schemes will keep arriving as the business grows. A first draft used a big if/else chain inside Order, and it's already getting hard to maintain.

Your Task

- Define a **PricingStrategy** interface with a single pricing method
- Implement **at least 3** concrete strategies, plus a default "no discount" strategy
- Give **Order** a PricingStrategy field it delegates total-calculation to
- Support swapping an order's strategy **at runtime** (e.g., a loyalty tier upgrade mid-session)
- Write 3–4 sentences: **why not just use if/else**, in your own words

Starter interface:

```
public interface PricingStrategy {
    double calculateTotal(double subtotal);
}
```

Milestone 2 — Observer: Live Order & Inventory Dashboards

Case Study

Every BrewHub order moves through states — queued, brewing, ready — and three different systems need to react the moment that happens: the in-store kitchen display, the customer's mobile app, and the inventory tracker. OrderService shouldn't need to know the details of any of them, and new subscribers should be easy to add later.

Your Task

- Define **Subject** and **Observer** interfaces (register, remove, notify)
- Implement **OrderStatusPublisher** as the concrete Subject
- Implement **at least 3** concrete observers: KitchenDisplay, CustomerNotifier, InventoryTracker
- Support **subscribing and unsubscribing** observers while the app is running
- Decide **push vs. pull**: do you send full order data, or let observers ask for it? Justify your choice

Milestone 3 — Decorator: Build-Your-Own Drink Pricing

Case Study

Customers can customize any base Beverage (Espresso, Drip) with condiments — extra shot, oat milk, vanilla, whipped cream — and combinations are unbounded. This is the same shape of problem Starbuzz Coffee solved in Chapter 3 with CondimentDecorator.

Your Task

- **Reuse or re-derive** the Beverage / CondimentDecorator hierarchy from Chapter 3
- Add **2 new condiments** and **1 new beverage** of your choosing
- Stack **at least 3 condiments** on one beverage and compute cost() correctly
- Make sure getDescription() **reflects every layer**, in the order added
- State one thing this design makes **easy** that subclassing every combination would not

Milestone 4 — Factory Method / Abstract Factory: Multi-Region Ingredient Sourcing

Case Study

BrewHub is opening roasting hubs in Seattle and Bogotá. Each hub sources its own beans, milk, and cups from local suppliers, and a hub must never accidentally combine, say, Seattle beans with Bogotá cups — that breaks a regional sourcing contract. This mirrors the NY/Chicago PizzaStore problem from Chapter 4.

Your Task

- Use **Factory Method** so each RoastingHub subclass creates its own Beans
- Use **Abstract Factory** to bundle a full family of parts — Beans, Milk, Cup — per region
- Show that swapping a hub's factory **cannot** produce a mismatched combination
- Add a **third region** of your choosing to prove the design extends cleanly
- Name the **design principle** (from Chapter 4) that the Abstract Factory enforces here

Milestone 5 — Singleton: One Central Order Ledger

Case Study

BrewHub needs exactly one OrderLedger tracking every paid transaction across all hubs for accounting reconciliation. Two ledger instances would silently double-count revenue — the same category of bug as this chapter's ChocolateBoiler story. BrewHub's backend runs each hub's checkout on its own thread.

Your Task

- Implement **OrderLedger** as a Singleton (private constructor + getInstance())
- Because checkout is **multithreaded**, getInstance() must be thread-safe
- Pick **one** of this chapter's three fixes — synchronized, eager, or double-checked locking
- Write a short justification: **why that fix**, given OrderLedger's likely call frequency
- Bonus: what would go wrong if OrderLedger were **not** a Singleton, concretely?

Starter skeleton:

```
public class OrderLedger {
    // your Singleton fields and getInstance() go here

    public synchronized void recordTransaction(double amount) {
        // append to the ledger
    }
}
```

Submission Checklist

- One package (or folder) **per milestone**, clearly labelled
- A short **README** per milestone explaining the design decision, not just the code
- Code **compiles and runs** a small demo/main() showing the pattern in action
- No milestone **duplicates logic** that an earlier milestone's pattern should own

Grading Rubric (100 points total)

Milestone	Pattern	Criteria	Points
1	Strategy	Interface + 3 strategies; Order delegates, no if/else chain	15
2	Observer	Subject/Observer interfaces; 3 observers; runtime subscribe/unsubscribe	20
3	Decorator	2 new condiments + 1 beverage; correct cost() and getDescription()	20
4	Factory / Abstract Factory	Factory Method per hub; Abstract Factory prevents mixed-region parts	25
5	Singleton	Thread-safe OrderLedger; written justification for the chosen fix	20

Stretch goal (optional, +10): wire Milestones 2 and 5 together — have OrderLedger itself subscribe as an Observer to OrderStatusPublisher, so it only records a transaction once an order reaches “ready.” This is a natural, realistic place where two patterns from this course meet in one design.